



Universidad
Indoamérica

UNIDAD 1 · INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN · 4.º SEMESTRE

Fundamentos de Sistemas Operativos

Andrés Rubio Proaño

Universidad Tecnológica Indoamérica · Facultad de Ingenierías

¿Qué es un sistema operativo?

Lo que vamos a hacer

- Principios generales
- Ejemplos de programación práctica (9 laboratorios en C)
- Estudiar las capas más bajas del software
- **Nuevo:** cómo los SO alojan agentes de IA

Lo que *no* es

- No es un curso de C → lee *The C Programming Language* (Kernighan y Ritchie)
- No es un curso de shell → aprende tu shell: usa `man`

Nueve unidades, un laboratorio por unidad

1	Introducción	TD0	6	Señales y memoria compartida	TD5
2	Llamadas al sistema	TD1	7	Hilos y concurrencia	TD6
3	Archivos	TD2	8	Redes: sockets	TD7
4	Procesos y memoria	TD3	9	SO nativos de IA	TD8
5	Tuberías	TD4			

Un sistema operativo...

- **administra y oculta** la complejidad del hardware al programador
- ofrece la interfaz de una **máquina virtual**: procesos, archivos, memoria virtual, sockets
- **protege** a los usuarios y a los programas entre sí

Los servicios del sistema operativo

Administrador de recursos

- Oculta la complejidad del hardware
- Resuelve conflictos de acceso y reparte con justicia
- Maneja los errores
- Impide el mal uso de la máquina

Facilita el uso de la máquina

- Bibliotecas estándar (glibc, gráficos, red)
- Programar se vuelve más simple y portable (POSIX)

¿Qué recursos? ¿Qué servicios?

Tiempo de CPU

Planificar tareas en los núcleos (U4)

Memoria

Asignación y protección (U4, U6)

Entrada/salida

Archivos y permisos, red, ventanas (U2, U3, U8)

Energía

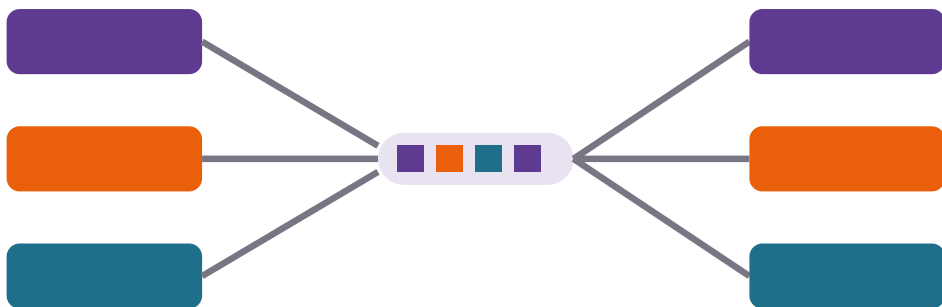
Frecuencia de la CPU, suspensión

Aceleradores (2026)

GPU y NPU para IA: el SO las reparte como cualquier recurso (U9)

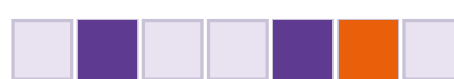
Estructurar los datos

Multiplexar la red

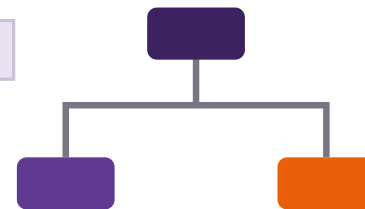


Un solo flujo de bits con datos de muchos programas hacia muchos servidores, que se separa al llegar.

Organizar el disco



bloques



/ carpetas y archivos

Un gran arreglo de bloques sobre el que se guarda un árbol de carpetas y archivos (U3).

En los supercomputadores (TOP500, junio de 2026)

#	SISTEMA	LUGAR	HPL
1	LineShine	Shenzhen, China	2,198 EF
2	El Capitan	LLNL, EE. UU.	1,809 EF
3	Frontier	ORNL, EE. UU.	1,353 EF
4	Aurora	ANL, EE. UU.	1,012 EF
5	JUPITER Booster	Jülich, Alemania	1,000 EF

1 EF = 10^{18} operaciones de punto flotante por segundo. Cinco sistemas ya superan el exaflop.

LineShine usa Kylin OS, una distribución basada en Linux. Fuente: top500.org (la lista se actualiza cada 6 meses).

En los centros de datos y en la nube

Centros de datos

Bancos, aseguradoras, universidades: miles de servidores, casi todos con Linux, muchos virtualizados.

Nube: sin ubicación física

Máquinas virtuales y contenedores repartidos por el mundo; el SO de tu "computador" es un servicio (almacenamiento, correo, cómputo).

En móviles, embebidos y sistemas críticos

- Teléfonos: Android (núcleo Linux) e iOS (núcleo XNU)
- Embebidos: módems, televisores, autos; SO de tiempo real como FreeRTOS, Zephyr, QNX o VxWorks

Sistemas críticos: a veces sin SO de propósito general

- Un solo programa que no necesita correr en otra parte
- Tiempo real: SO específico para un conjunto de tareas y eventos, o ninguno
- Lenguajes síncronos que garantizan tiempos de reacción: Lustre, Esterel

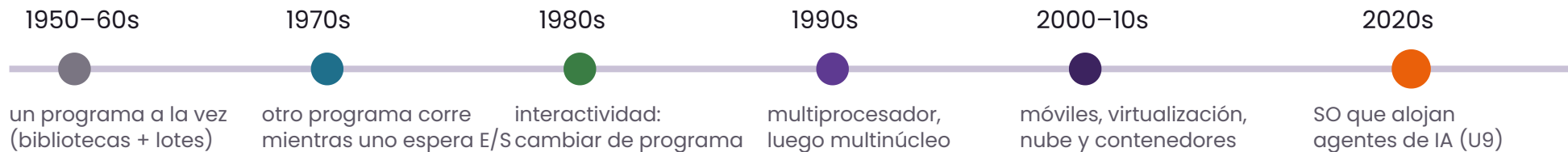
Ocultar la complejidad del hardware

Las máquinas son muy distintas:

- tipo y número de procesadores y núcleos (este equipo: 28)
- cantidad de memoria
- tipo de disco o de tarjeta de red

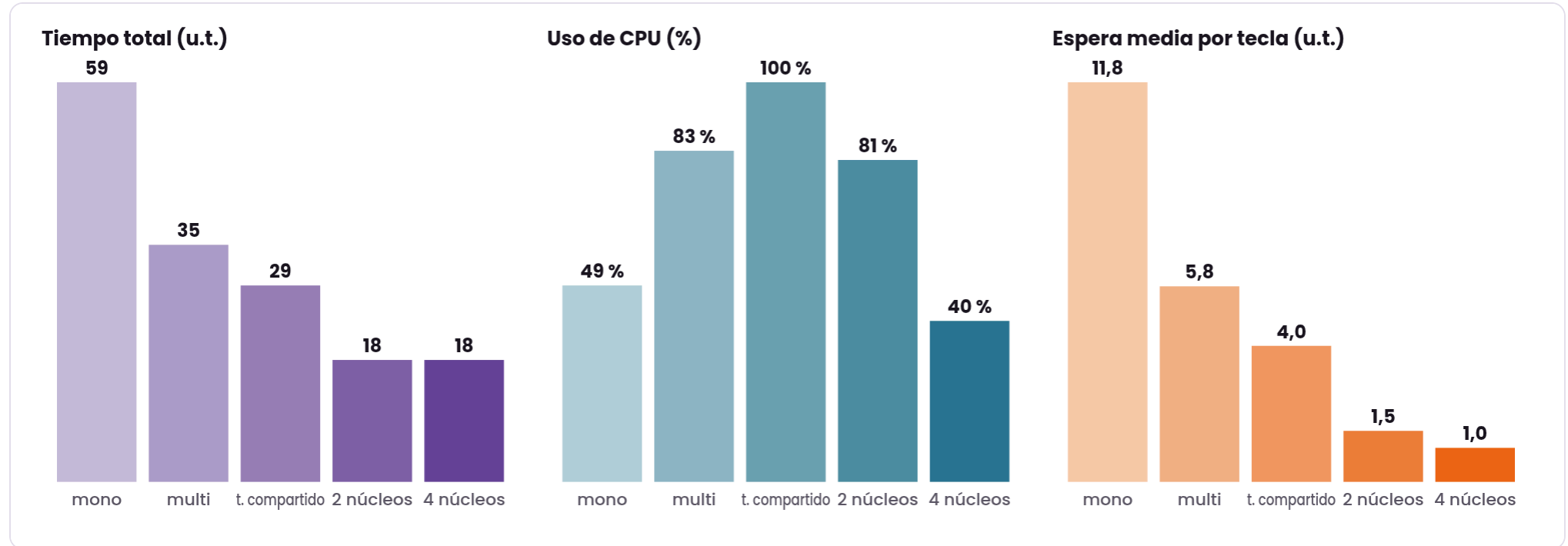
Queremos que **el mismo programa funcione en todas** sin cambios: con más o menos memoria, con cualquier tarjeta de red, con un SSD local o un NAS remoto.

Un poco de historia

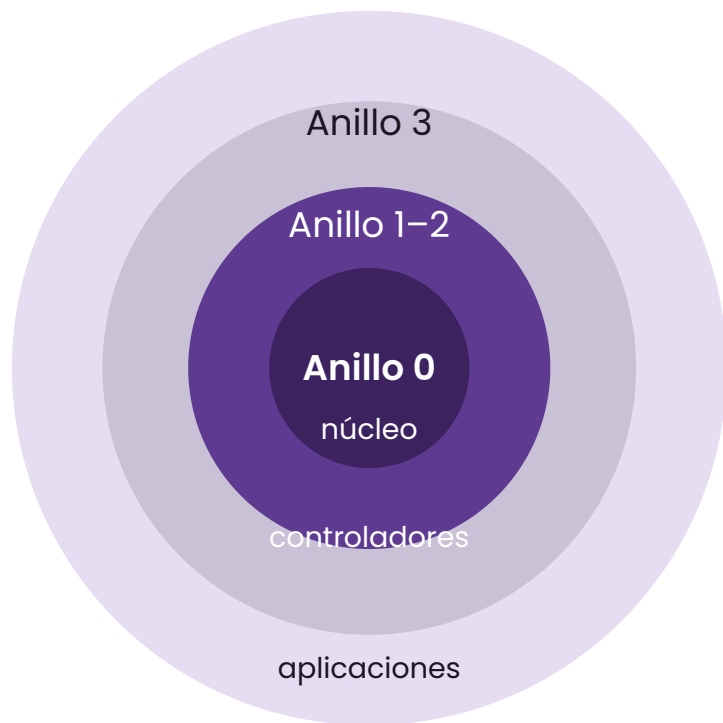


Al principio un SO era apenas unas bibliotecas, con dos supuestos: *ningún programador malicioso* y *un programa a la vez*. Consecuencia: tiempo de máquina y de usuario desperdiciado. Cada época quitó un supuesto... y agregó un problema que este curso estudia.

Tiempo de máquina y de usuario desperdiciado



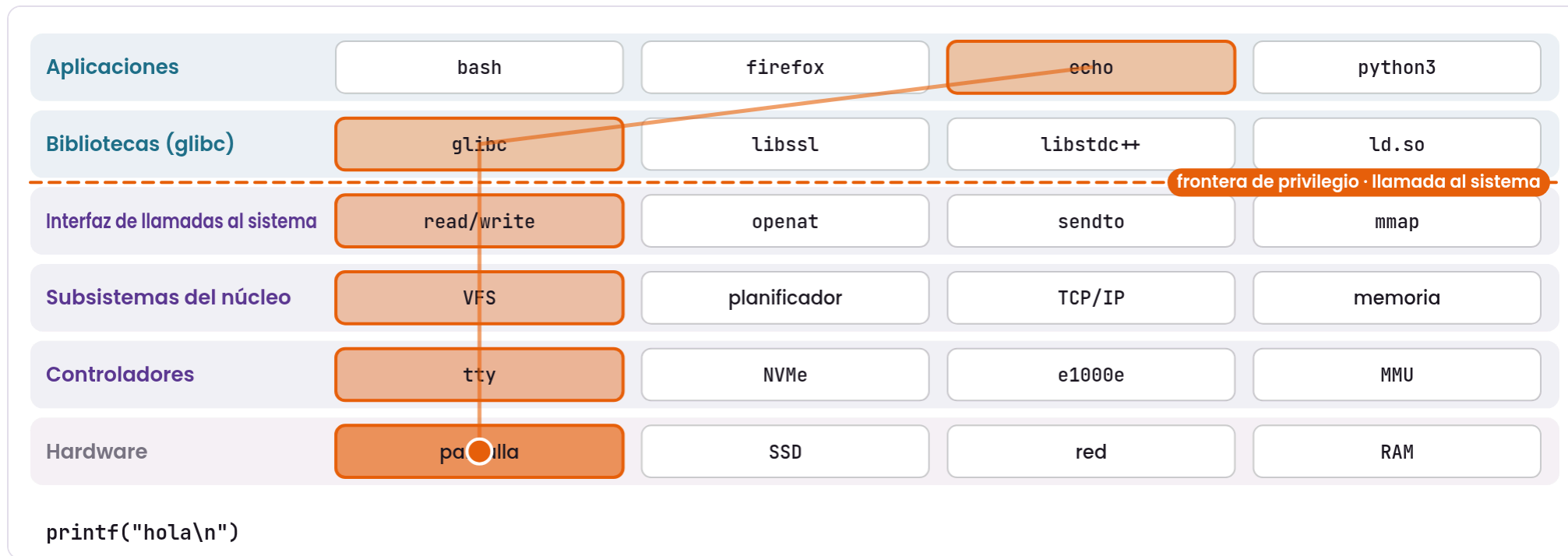
Estructura de un SO moderno: una cebolla



x86 define 4 anillos; Linux y Windows usan solo dos: **anillo 0** (núcleo, más privilegiado) y **anillo 3** (aplicaciones, menos privilegiado).

El núcleo **no es root**: root es un usuario; el núcleo es el código que corre en el anillo 0.

Los componentes, recorridos por una petición

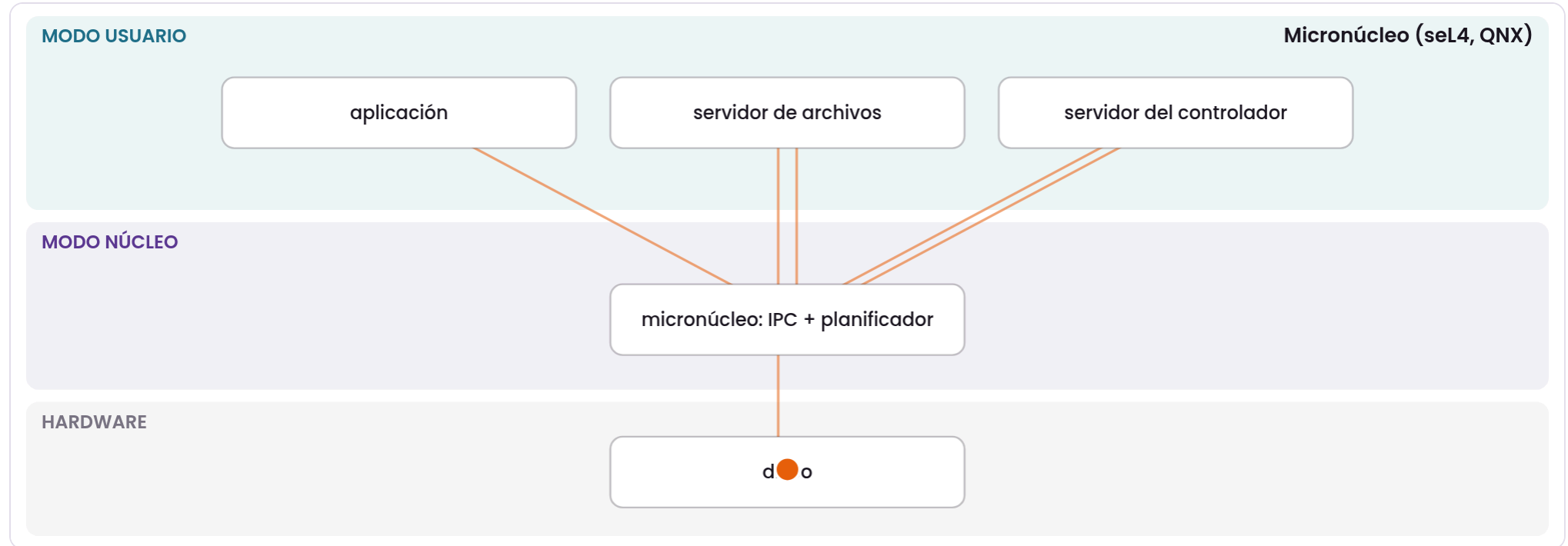


Un software muy complejo

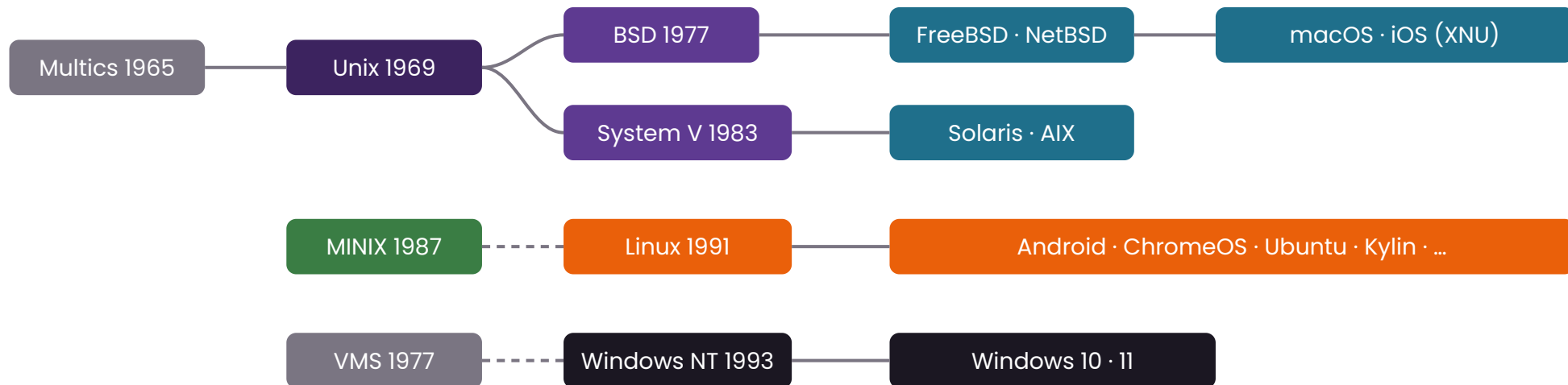
- El núcleo Linux tiene decenas de millones de líneas de código; la mayor parte son **controladores**.
- Miles de desarrolladores por versión; una versión nueva cada 9–10 semanas.
- En el equipo del curso: Linux 7.0 y **258 módulos** cargados.

Por eso importa la arquitectura: ¿qué pasa si falla uno de esos controladores?

Monolítico, micronúcleo, híbrido

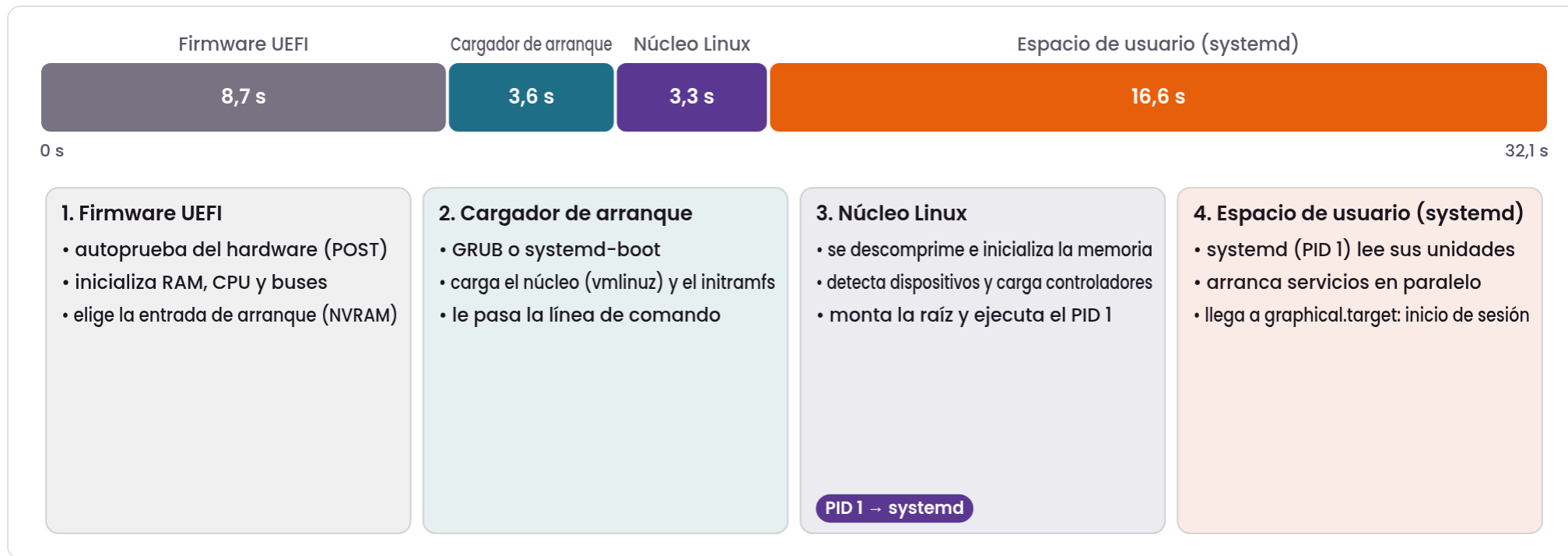


Ejemplos y genealogía



Línea continua: descendencia de código; punteada: influencia de diseño (Linux se inspiró en MINIX; Windows NT, en VMS, con el mismo arquitecto principal). También: GNU/Hurd (micronúcleo), Plan 9, VxWorks.

Del encendido al inicio de sesión (tiempos reales)



Para leer más

- Silberschatz, Galvin, Gagne. *Operating System Concepts*, 10th ed.
- Arpaci-Dusseau. *Operating Systems: Three Easy Pieces* (libre)
- Cursos en línea: CS 162 (Berkeley), CS 140 (Stanford)
- top500.org · man7.org/linux/man-pages · docs.kernel.org



Universidad
Indoamérica

PARA LLEVAR · LABORATORIO TD0

Tres preguntas

1. ¿Qué servicios da un SO como administrador de recursos?
2. ¿Por qué el núcleo no es lo mismo que root?
3. ¿Qué gana y qué pierde un micronúcleo frente a Linux?